Backend Development with FastAPI

Mandatory Requirements

All projects must comply with the following requirements:

---

1. Project Structure

Students must organize their project using a clean and maintainable structure. The codebase should be modular, properly separated into components such as routes, models, schemas, and services.

---

2. RESTful API Implementation

The application must implement a fully functional RESTful API, including the following operations:

- **GET**
    - Retrieve all records
    - Retrieve a single record by ID
- **POST**
    - Create new records
- **PUT**
    - Update existing records
- **DELETE**
    - Remove records

Additionally, students must ensure:

- Proper use of HTTP status codes
- Request validation using Pydantic models
- Use of response models for consistent output formatting

---

3. JWT Authentication

The system must include secure authentication using JSON Web Tokens (JWT), with the following features:

- User registration
- User login
- JWT token generation
- Token validation
- Protection of secured routes using authentication

---

4. Role-Based Authorization

The application must implement role-based access control with at least two roles, such as:

- **Admin**
- **User**

Access to endpoints must be restricted based on roles. For example:

- Admin users can perform delete operations
- Regular users have limited access (e.g., read-only)

---

5. Error Handling

The project must include proper error handling mechanisms:

- Use of HTTPException
- Clear and descriptive error messages
- Handling of validation errors

---

6. Git & GitHub Version Control

The project must be managed using Git and hosted on GitHub with proper collaboration practices:

- All team members must contribute to the repository with **meaningful and traceable commits**

- The repository must reflect **individual contributions** from each team member (evaluated via commit history, pull requests, and branches)
- Students must follow a **branching strategy** (e.g., main, develop, feature branches per member)
- The repository must include:
    - A clear README.md with project description, setup instructions, and team member roles

⚠️ ***Note:*** *Repositories with commits from a single member or bulk/last-minute commits will result in a penalty. Each member's contribution must be clearly visible.*

---

7. Caching using Redis

The application must implement a caching layer using **Redis** to improve performance:

- Cache frequently accessed data (e.g., GET all records, GET by ID)
- Implemnt Cache-Aside Pattern.
- Implement **cache invalidation** strategies when data is created, updated, or deleted
- Demonstrate measurable **performance improvement** through caching (e.g., reduced response times)

---

8. Logging and Monitoring Dashboard

The project must implement a comprehensive logging and monitoring system:

Logging:

- Implement structured logging throughout the application using Python's logging module or a library such as loguru
- Log levels must be used appropriately (DEBUG, INFO, WARNING, ERROR, CRITICAL)
- Logs must capture:
    - Incoming API requests and responses (method, endpoint, status code, response time)
    - Authentication events (login attempts, failures, token validations)

    - Error and exception details
    - CRUD operations and database interactions

Monitoring Dashboard:

- Integrate a monitoring/visualization tool to display application metrics and logs. Acceptable options include:
    - **Prometheus + Grafana**
    - **ELK Stack (Elasticsearch, Logstash, Kibana)**
    - **Custom-built dashboard** (using FastAPI endpoints + a simple frontend)
    - Or any equivalent monitoring solution
- The dashboard must display at minimum:
    - API request counts and response times
    - Error rates and recent error logs
    - System health status

---

9. API Testing

The project must include a comprehensive test suite covering the application's core functionality:

- Use **pytest** as the testing framework
- Use **TestClient** from FastAPI for HTTP-level testing
- Tests must cover:
    - **Authentication functionality** (registration, login, token generation and validation)
    - **Protected endpoints** (verify unauthorized access is blocked, role-based restrictions are enforced)
    - **CRUD operations** for all major entities
    - **Business logic validation** (e.g., preventing invalid state transitions, enforcing borrowing limits, preventing double bookings — depending on the project)

    - **Edge cases and error handling** (invalid inputs, missing resources, duplicate entries)
- Tests must be organized in a dedicated tests/ directory with clear naming conventions

---

Bonus Features (Optional)

Frontend Integration

- Develop a simple frontend using:
    - HTML / CSS / JavaScript, **OR**
    - React, Vue, or any other frontend framework
- The frontend must communicate with the FastAPI backend
- Must demonstrate at least:
    - User login/registration
    - Displaying data from the API
    - Performing CRUD operations through the UI

Docker Integration

- Create a Dockerfile for the FastAPI application
- Use docker-compose to orchestrate multiple services (e.g., FastAPI app, database, Redis)
- Ensure the entire application stack runs successfully in a Docker environment
- Include clear instructions in the README.md for building and running the containers

Project Details

Project 1: Student Management System

Description:

Develop a backend system to manage university students with secure access control and advanced querying capabilities.

Entities

- Users
- Students

Features

- User registration and authentication
- Full CRUD operations for students
- Advanced filtering (e.g., department, GPA)
- Pagination support
- Students can only access their own profile
- Audit logging for updates

Roles

Admin: Full control over student records

Student: View and partially update own profile

Project 2: Library Management System

Description

Build a backend API for managing a library system with borrowing and tracking functionality.

Entities

- Users
- Books
- BorrowRecords

Features

- CRUD operations for books
- Borrow and return system with availability validation
- Prevent borrowing unavailable books
- Track borrowing history
- ・Limit number of borrowed books per user

Roles

- Admin (Librarian): Manage books and view all records
- Member: Borrow, return, and view personal history

Project 3: E-Commerce System

Description

Develop a scalable backend system for an online store with product and order management.

Entities

- Users
- Products
- Categories
- Orders
- OrderItems

Features

- Product and category management (CRUD)
- Product search, filtering, and pagination
- Shopping cart functionality
- Order placement and tracking
- Basic inventory management (stock updates)

Roles

Admin: Manage products and categories, view all orders

Customer: Browse products, manage cart, and place orders

Project 4: Blog Management System

Description

Create a backend system for a blogging platform with structured content management.

Entities

- Users
- Posts
- Comments

Features

- CRUD operations for posts and comments
- Ownership-based access control (authors manage their own content)
- Nested comments support
- Pagination for posts and comments

Roles

Admin: Full moderation control

Author: Create and manage own posts

Reader: View and comment

Project 5: Task Management System

Description

Develop a backend system for managing projects and tracking tasks with workflow validation.

Entities

- Users
- Projects
- Tasks

Features

- Project and task management

- Task assignment to users
- Task status lifecycle (To Do → In Progress → Done)
- Enforce valid status transitions
- Filter tasks by status, priority, and assignee

Roles

Admin: Manage projects and tasks

Project Manager: Assign and monitor tasks

Employee: Update assigned tasks only

Project 6: Hospital Appointment System

Description

Build a backend system for managing hospital appointments between patients and doctors.

Entities

- Users
- Doctors
- Patients
- Appointments

Features

- Manage doctors and patients
- Book and cancel appointments
- Prevent double booking
- View schedules by doctor and patient
- Appointment status management (Scheduled, Completed, Cancelled)

Roles

Admin: Manage doctors and patients

Doctor: View and update appointment status

Patient: Book and manage appointments

Project 7: Online Examination System

Description

Create a backend system for conducting and managing online examinations.

Entities

- Users
- Exams
- Questions
- StudentAnswers
- Results

Features

- Create and manage exams and questions
- Support timed exams
- Automatic grading for objective questions
- Store and retrieve results
- Basic analytics (scores and averages)

Roles

Admin: Manage exams and view all results

Student: Take exams and view results